CHAPTER 07

DynamoDB

Sessions · Cart · Order Events · Streams · GSI · DAX · Global Tables

07-00 Chapter Overview — DynamoDB at ShopFlow

ShopFlow uses DynamoDB for three high-throughput use-cases where Aurora's relational model adds unnecessary overhead: session storage (1M concurrent users), shopping cart (atomic updates with TTL), and order event log (append-only, Kinesis-sourced). Each table uses On-Demand billing for Black Friday resilience.

Use-Case	Table Name	Partition Key	Sort Key	Access Pattern
User sessions	shopflow-sessions	userId (UUID)	sessionId	All sessions per user; lookup by token (GSI)
Shopping cart	shopflow-cart	userId	productId	All cart items per user; atomic qty updates
Order events	shopflow-order-events	orderId	ts#eventId	Chronological event history; by-user GSI
Product cache	shopflow-product-cache	productId	—	Single-item get; 1hr TTL auto-expire
Rate limiting	shopflow-rate-limit	ip#endpoint	window	Sliding window counter; TTL cleanup

Metric	shopflow-sessions	shopflow-cart	shopflow-order-events
Normal peak	1,000 RCU/s + 1,000 WCU/s	200 RCU/s + 500 WCU/s	100 WCU/s (event-driven)
Black Friday peak	10,000 RCU/s + 10,000 WCU/s	2,000 RCU/s + 5,000 WCU/s	1,000 WCU/s
Item size (avg)	0.5 KB	0.3 KB	1.2 KB (with items array)
Billing mode	On-Demand (PAY_PER_REQUEST)	On-Demand	On-Demand
TTL attribute	expiresAt (24hr)	abandonedAt (7 days)	None — retain forever

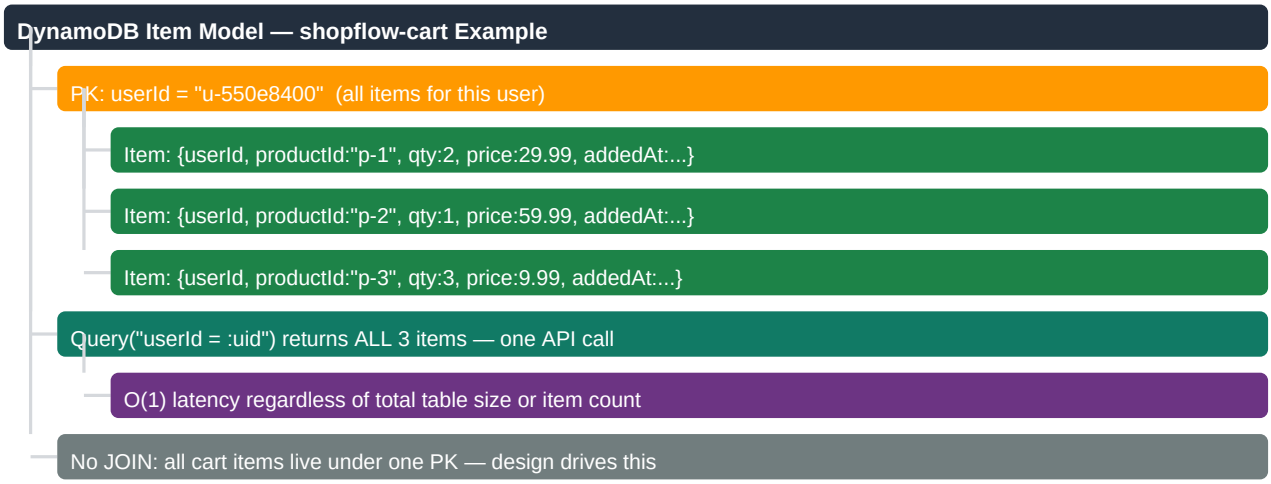
■ DynamoDB On-Demand scales to any traffic level with zero pre-warming required — within the soft limit of 2x previous peak per 30 minutes. At ShopFlow's normal peak of 10,000 RCU/s, Black Friday can spike to 20,000 RCU/s instantly. Beyond that, capacity doubles every 30 minutes. Load-test 48 hours before Black Friday to establish baseline.

07-01 DynamoDB Core Concepts — Keys, Items & Attributes

DynamoDB stores Items (documents) in Tables. Every item has a primary key: partition key alone (simple PK) or partition key + sort key (composite PK). All other attributes are schema-less — different items in the same table can have completely different attributes.

Concept	Relational Equivalent	DynamoDB Specifics
Table	Table (no FK constraints)	Single collection of items. No schema enforcement.
Item	Row	JSON-like document. Max 400KB per item including attribute names.
Partition Key (PK)	Part of Primary Key	Hashed to determine physical storage partition.
Sort Key (SK)	Part of Primary Key	Orders items within one PK. Enables range queries.
Attribute	Column	Any type: S, N, B, SS, NS, BS, L (list), M (map), NULL, BOOL.
GSI (Global Secondary Index)	Non-PK index	Separate read capacity; eventual consistency default.
LSI (Local Secondary Index)	Alt sort on same PK	Must define at table creation; shares table capacity.

Concept	Relational Equivalent	DynamoDB Specifics
TTL attribute	Cron-based delete job	Number (epoch seconds): auto-deletes within 48 hours of expiry.
Condition expression	WHERE in UPDATE	Optimistic lock: ConditionExpression="attribute_not_exists(id)".
TransactWrite	BEGIN/COMMIT	Up to 100 items across tables; 2x WCU cost.



■ DynamoDB schema design rule: identify all access patterns before writing any code. List every query the application will execute. Design PK and SK to answer the most frequent queries directly. If a pattern requires filtering millions of items, the data model is wrong — redesign keys. Access pattern-first is the opposite of relational design.

07-02 Partition Key Design & Hot Partition Prevention

DynamoDB distributes items across physical partitions based on a hash of the partition key. A hot partition occurs when most writes go to one PK value — saturating one partition while others are idle. High-cardinality partition keys and write sharding prevent hot partitions at any scale.

Anti-Pattern PK	Problem	Better PK	Why Better
date: "2024-11-29"	All Black Friday writes hit one partition	userId (UUID)	1M users = 1M partitions, even distribution
status: "PENDING"	3 statuses = max 3 partitions in use	orderId (UUID)	Each order = unique partition
country: "US"	90% of traffic to "US" partition	userId#country	Prefix with high-cardinality key
sequential BIGINT	First 50% of orders on first partition	gen_random_uuid() equivalent	Random hash distributes evenly

Write Sharding Pattern — High-Volume Counter

```
// Problem: ProductView counter for 1 product hits same PK 10,000/s
// One partition max: ~1,000 WCU/s → throttled at 10,000/s
// Solution: shard across 10 items, scatter writes
int shard = ThreadLocalRandom.current().nextInt(10); // 0-9
String pk = "view#" + productId + "#" + shard; // "view#p-1#3"
dynamo.updateItem(UpdateItemRequest.builder()
.tableName("shopflow-product-cache")
.key(Map.of("productId", AttributeValue.fromS(pk)))
.updateExpression("ADD #viewCount :one")
.expressionAttributeNames(Map.of("#viewCount","viewCount"))
.expressionAttributeValues(Map.of(":one",AttributeValue.fromN("1")))
.build());
// Read: sum all 10 shards to get total view count
List<String> shardKeys = IntStream.range(0,10)
.mapToObj(i -> "view#" + productId + "#" + i)
```

```
.collect(toList());
// BatchGetItem across 10 PKs, sum viewCount attributes
```

Traffic Level	Unsharded Partition	10 Shards	100 Shards
1,000 WCU/s	OK (1,000 per partition)	100 per shard — ample headroom	10 per shard
10,000 WCU/s	THROTTLED (10x partition limit)	1,000 per shard — OK	100 per shard — OK
100,000 WCU/s	THROTTLED	THROTTLED	1,000 per shard — OK

■ A single DynamoDB partition can sustain up to 1,000 WCU/s and 3,000 RCU/s. These limits apply per physical partition, not per table. On-Demand tables still have per-partition limits — distribute writes evenly across many PK values to achieve the advertised unlimited scale.

07-03 Sessions Table — Design, CDK & TTL

ShopFlow session storage: 1M concurrent sessions, 24-hour TTL, single-table design with GSI for token-based auth lookup. DynamoDB replaces Redis for session storage at this scale — no cluster management, automatic failover, and native TTL without manual cron jobs.

Session Item Schema

```
{
"userId": "u-550e8400-e29b-41d4-a716-446655440000", // PK
"sessionId": "s-f47ac10b-58cc-4372-a567-0e02b2c3d479", // SK
"sessionToken":"st-9b2d5a3f1c6e...", // GSI PK — auth middleware
"deviceType": "mobile",
"ipAddress": "192.168.1.1",
"createdAt": 1732886400,
"expiresAt": 1732972800, // TTL: epoch seconds — auto-deleted in 48h
"cartItemCount": 3,
"lastActivity":"2024-11-29T14:00:00Z"
}
```

Sessions Table CDK

```
const sessionTable = new dynamodb.Table(this,"ShopFlowSessions",{
tableName: "shopflow-sessions",
partitionKey: { name:"userId", type:dynamodb.AttributeType.STRING },
sortKey: { name:"sessionId", type:dynamodb.AttributeType.STRING },
billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
timeToLiveAttribute: "expiresAt",
pointInTimeRecovery: true,
encryption: dynamodb.TableEncryption.AWS_MANAGED,
removalPolicy: RemovalPolicy.RETAIN,
});
// GSI: auth middleware validates token per API request
sessionTable.addGlobalSecondaryIndex({
indexName: "sessionToken-index",
partitionKey: { name:"sessionToken", type:dynamodb.AttributeType.STRING },
projectionType: dynamodb.ProjectionType.ALL,
});
```

Operation	API Call	Consistency	RCU Cost
Validate session (auth middleware)	GetItem(userId + sessionId)	Strongly consistent	1 RCU (0.5KB item rounds to 4KB)
Auth by token (login flow)	Query on sessionToken-index GSI	Eventually consistent	0.5 RCU
Extend session TTL	UpdateItem (set expiresAt)	—	1 WCU
Delete session (logout)	DeleteItem(userId + sessionId)	—	1 WCU
Get all user sessions	Query(userId)	Eventually consistent	1 RCU per 4KB of results

■ TTL deletion is not instant — items are removed within 48 hours of the TTL epoch value. ALWAYS add FilterExpression="expiresAt > :now" to session queries. Do not rely on TTL as a real-time expiry mechanism. For immediate invalidation (logout, password change), explicitly delete the session item and/or add to a DynamoDB revocation table.

07-04 Shopping Cart Table — Atomic Updates & Conditional Writes

Cart updates from multiple browser tabs, mobile apps, and race conditions require atomic operations. DynamoDB UpdateItem with ADD performs atomic increment/decrement — concurrent operations queue internally without overwriting each other. ConditionExpressions prevent negative quantities.

Cart Table CDK

```
const cartTable = new dynamodb.Table(this,"ShopFlowCart",{
tableName: "shopflow-cart",
partitionKey: { name:"userId", type:dynamodb.AttributeType.STRING },
sortKey: { name:"productId", type:dynamodb.AttributeType.STRING },
billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
timeToLiveAttribute: "abandonedAt", // 7-day auto-cleanup
stream: dynamodb.StreamViewType.NEW_IMAGE, // cart analytics
pointInTimeRecovery: true,
});
```

Atomic Cart Operations — Add, Remove, Clear

```
// ADD 1 to quantity (atomic — concurrent adds never lost)
dynamo.updateItem(UpdateItemRequest.builder()
.tableName("shopflow-cart")
.key(Map.of("userId", AttributeValue.fromS(userId),
"productId", AttributeValue.fromS(productId)))
.updateExpression(
"ADD #qty :one SET #addedAt = if_not_exists(#addedAt, :now)")
.expressionAttributeNames(Map.of("#qty","qty","#addedAt","addedAt"))
.expressionAttributeValues(Map.of(":one",AttributeValue.fromN("1"),
":now",AttributeValue.fromS(now())))
.build());
// REMOVE 1 from quantity (conditional: prevent qty going below 1)
dynamo.updateItem(UpdateItemRequest.builder()
.tableName("shopflow-cart")
.key(Map.of("userId", AttributeValue.fromS(userId),
"productId", AttributeValue.fromS(productId)))
.updateExpression("ADD #qty :minus")
.conditionExpression("#qty >= :one")
.expressionAttributeValues(Map.of(":minus",AttributeValue.fromN("-1"),
":one", AttributeValue.fromN("1")))
.build());
```

Operation	DynamoDB API	Atomicity	Use Case
Add item to cart	UpdateItem + ADD qty	Single-item atomic	Concurrent tab/device add-to-cart
Remove one item	UpdateItem + condition qty>=1	Atomic with check	Prevent negative quantities
Checkout (clear cart)	TransactWrite + Delete all	ACID multi-item	Atomically move cart to order
View full cart	Query(userId)	Eventually consistent	Render cart page — all items

■ Never PutItem to update cart quantity. PutItem replaces the entire item — if two tabs read qty=2 and both PutItem with qty=3, one update is silently discarded. Use UpdateItem with ADD for numeric attributes. ADD is idempotent from the perspective of lost updates — all concurrent ADD operations apply and none are dropped.

07-05 Order Events Table — Composite SK & GSI Design

The order-events table uses an overloaded sort key format "YYYYMMDDTHHMMSS#eventId" to enable chronological queries per order. A GSI on userId allows the order history page to fetch all orders without a full table scan. Sparse GSI on pendingOrderId indexes only active orders.

Order Event Item Schema

```
{
"orderId": "ord-7f3e2a1b", // PK
"sk": "20241129T140000#evt-001", // SK = timestamp#eventId
"eventType": "ORDER_PLACED",
"userId": "u-550e8400", // GSI PK for order history
"status": "PENDING",
"totalAmt": 149.97,
"items": [{"productId":"p-1","qty":2,"price":29.99}],
"timestamp": "2024-11-29T14:00:00Z"
}
```

Order Events CDK with GSI

```
const orderEventsTable = new dynamodb.Table(this,"ShopFlowOrderEvents",{
tableName: "shopflow-order-events",
partitionKey: { name:"orderId", type:dynamodb.AttributeType.STRING },
sortKey: { name:"sk", type:dynamodb.AttributeType.STRING },
billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
pointInTimeRecovery: true,
stream: dynamodb.StreamViewType.NEW_AND_OLD_IMAGES,
});
orderEventsTable.addGlobalSecondaryIndex({
indexName: "userId-ts-index",
partitionKey: { name:"userId", type:dynamodb.AttributeType.STRING },
sortKey: { name:"sk", type:dynamodb.AttributeType.STRING },
projectionType: dynamodb.ProjectionType.INCLUDE,
nonKeyAttributes: ["orderId","status","totalAmt","timestamp"],
});
```

Query Pattern	KeyCondition + Filter	Index	ScanIndexForward
All events for order (oldest first)	orderId=:oid	Table PK	true (default)
Last 10 events for order	orderId=:oid	Table PK + SK	false, Limit=10
Order history for user (last 20 orders)	userId=:uid	GSI userId-ts-index	false, Limit=20
Find ORDER_PLACED events only	orderId=:oid + SK begins_with "ORDER_PLACED#"	Table PK	—

■ FilterExpression in DynamoDB is applied AFTER items are read from the table — it does not reduce RCU consumption. To query ORDER_PLACED events efficiently, encode the event type in the sort key prefix: "ORDER_PLACED#20241129T140000#evt-001". Then KeyConditionExpression="sk begins_with :prefix" reads only ORDER_PLACED events efficiently.

07-06 RCU, WCU & On-Demand Capacity Planning

1 RCU = 1 strongly consistent read of up to 4KB per second (or 2 eventually consistent reads). 1 WCU = 1 write of up to 1KB per second. Items larger than these limits round up to the next unit. On-Demand charges per actual request — no pre-provisioning required.

Item Size	Strongly Consistent RCU	Eventually Consistent RCU	Write WCU
0.5 KB	1 RCU (rounds up to 4KB)	0.5 RCU	1 WCU (rounds up to 1KB)
4 KB	1 RCU	0.5 RCU	4 WCU
6 KB	2 RCU	1 RCU	6 WCU

Item Size	Strongly Consistent RCU	Eventually Consistent RCU	Write WCU
1.2 KB (order event with items)	2 RCU	1 RCU	2 WCU

Black Friday Capacity Calculation

```
// shopflow-sessions: 10,000 req/s (auth check on every API request)
// GetItem 0.5KB item → 1 RCU strongly consistent
// UpdateItem expiresAt → 1 WCU
// RCU/s: 10,000 WCU/s: 10,000
// Daily (8h BF peak): 10K × 3600s × 8h = 288M RCU + 288M WCU
// On-Demand cost: 288M × $0.25/M + 288M × $1.25/M = $72 + $360 = $432 (BF day)
// shopflow-cart: 1,000 add-to-cart/s + 500 view-cart/s
// UpdateItem 0.3KB → 1 WCU; Query 5 items × 0.3KB = 1.5KB → 1 RCU
// WCU/s: 1,000 RCU/s: 500
// Daily (8h): 29M WCU + 14M RCU
// Cost: 29M × $1.25/M + 14M × $0.25/M = $36 + $3.50 = $39.50 (BF day)
```

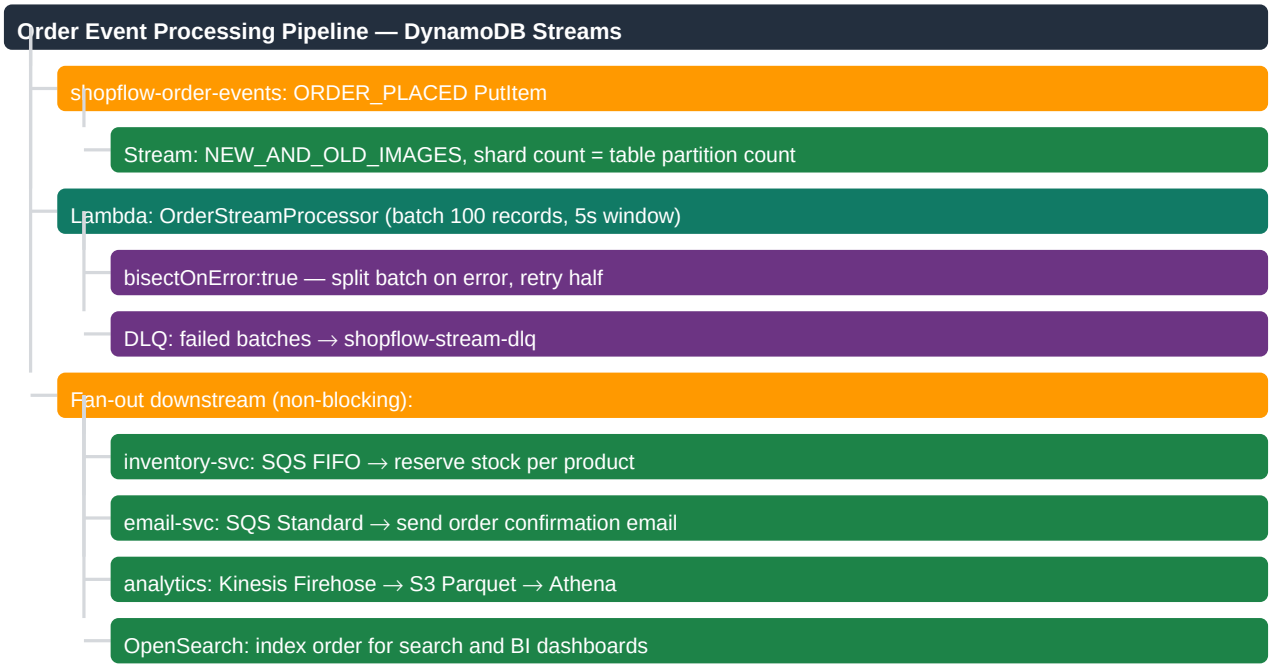
On-Demand vs Provisioned Decision Matrix

Criteria	On-Demand (PAY_PER_REQUEST)	Provisioned + Auto Scaling
Traffic pattern	Spiky, unpredictable (Black Friday)	Steady, predictable (>60% utilization)
Provisioning risk	None — unlimited scaling	Must pre-warm for spikes; 15-60min ramp
Cost at high utilization	~30% more expensive per unit	Cheaper (no unused capacity charge)
Cold table (no history)	Any capacity from first request	Limited to previous 5-min peak × 2
ShopFlow production	All tables (Black Friday resilience)	Dev/test tables with known load

■ On-Demand tables scale to 2× previous peak capacity instantly, then double every 30 minutes. A table with no traffic history that receives 10,000 WCU/s immediately will be throttled — it can only handle its initial capacity ceiling. Solution: run a pre-Black Friday load test (48h before) to establish baseline capacity that On-Demand can then scale from.

07-07 DynamoDB Streams & Lambda Event Processing

DynamoDB Streams capture item-level changes in order. ShopFlow uses streams on order-events to trigger inventory reservation, email dispatch, and analytics ingestion — replacing synchronous API calls with event-driven fan-out. Streams retain records for 24 hours.



Stream Lambda CDK

```
const streamLambda = new lambda.Function(this,"OrderStreamProcessor",{
runtime: lambda.Runtime.JAVA_21,
handler: "com.shopflow.OrderStreamHandler::handleRequest",
code: lambda.Code.fromEcr(ecrRepo),
timeout: Duration.minutes(5),
memorySize: 512,
reservedConcurrentExecutions: 10,
});
streamLambda.addEventSource(new DynamoEventSource(orderEventsTable,{
startingPosition: lambda.StartingPosition.LATEST,
batchSize: 100,
bisectOnError: true,
retryAttempts: 3,
onFailure: new SqsDestination(dlqQueue),
}));
```

Stream View Type	Records Sent	Best Use Case	Cost
KEYS_ONLY	PK + SK only	Cache invalidation by key	Cheapest — minimal data transfer
NEW_IMAGE	Item after change	Event-driven processing	Medium
OLD_IMAGE	Item before change	Audit log, undo buffer	Medium
NEW_AND_OLD_IMAGES	Before + after both	Change data capture, diff detection	Highest — 2x data

■ DynamoDB Streams retain records for only 24 hours. If the Lambda consumer is paused (deployment, throttling, sustained errors) for more than 24 hours, stream records are permanently lost. For critical events, additionally write to SQS (7-day retention) or Kinesis (365-day retention) as a durable buffer.

07-08 Global Secondary Index Design Patterns

GSIs enable queries on non-PK attributes. Three advanced patterns: sparse indexes (only items with the attribute indexed), overloaded keys (multiple entity types on one GSI), and INCLUDE projections (replica only needed attributes). GSI writes consume additional WCUs — account for write amplification.

Pattern	Description	ShopFlow Application	Benefit
Sparse GSI	Items without the GSI key excluded automatically	pendingOrderId GSI — only PENDING orders indexed	Smaller index, no need to filter comp
Overloaded key	GSI PK = "entityType#id" for multiple entity types	GSI PK="userId" serves both orders and profiles	One GSI for multiple query patterns
INCLUDE projection	Only specified attributes replicated to GSI	Order history: include orderId, status, totalAmt only	80% smaller GSI — fewer WCUs on
ALL projection	Full item in GSI — no table read needed	Sessions: ALL projection avoids second table read	Zero table reads on GSI queries
KEYS_ONLY projection	Only PK/SK/GSI keys replicated	Cache invalidation GSI — only need key to delete	Minimal storage, fastest GSI writes

GSI Write Amplification Calculation

```
// Table with 3 GSIs, 1,000 WCU/s base writes
// Each table write also writes to GSIs where item has GSI key attributes
// Effective consumption: 1,000 × (1 base + 3 GSIs) = 4,000 WCU/s
// Sparse GSI: item only has pendingOrderId when status=PENDING
// 10% of orders are PENDING → only 10% of writes hit pendingOrderId GSI
// Effective: 1,000 × (1 base + 1 order-history + 0.1 sparse) = 2,100 WCU/s
// Savings: 1,900 WCU/s vs 4,000 WCU/s = 52% reduction
```

GSI	Table	GSI PK	GSI SK	Projection
sessionToken-index	sessions	sessionToken	—	ALL — full item needed for auth
userId-ts-index	order-events	userId	sk	INCLUDE: orderId,status,totalAmt,ts
category-status-index	product-cache	categoryId	updatedAt	ALL — full product needed
pendingOrderId-index	order-events	pendingOrderId	—	INCLUDE: userId,totalAmt (sparse)

■ GSI updates are asynchronous — there is a small propagation delay (milliseconds to seconds) between a table write and the GSI update. For use-cases where stale GSI data would cause incorrect behavior (e.g., session token auth), either use strongly consistent reads from the table PK or add application-level consistency checks after writes.

07-09 Single-Table Design Pattern

Single-table design stores multiple entity types (orders, order items, users, products) in one DynamoDB table using generic PK/SK attribute names. The benefit: a single Query can retrieve a user + all their orders + all order items in one API call, eliminating N+1 patterns entirely.

Entity Model — Generic PK/SK

```
// All entities use PK and SK — generic names
// Entity type encoded in SK prefix
// User entity: PK="USER#u-123" SK="PROFILE"
// { PK, SK, userId, email, name, createdAt }
// Order entity: PK="USER#u-123" SK="ORDER#ord-456"
// { PK, SK, orderId, status, totalAmt, createdAt }
// Order item: PK="ORDER#ord-456" SK="ITEM#p-789"
// { PK, SK, productId, qty, price }
// Access patterns:
// 1. User profile + all orders in one Query:
// Query(PK="USER#u-123") → returns PROFILE + all ORDER#xxx items
// 2. All items for one order:
// Query(PK="ORDER#ord-456") → returns all ITEM#xxx items
// 3. Filter to orders only:
// Query(PK="USER#u-123", SK begins_with "ORDER#")
```

Single-Table vs Multi-Table — ShopFlow Decision

Factor	Single-Table	Multi-Table (ShopFlow choice)	ShopFlow Rationale
Query efficiency	One request for related entities	Multiple requests (N+1 risk)	Teams understand multi-table; explicit queries
Schema clarity	Hard to read — generic PK/SK	Clear — table name = entity type	Onboarding new engineers in 1 day not 1 wee
Access pattern changes	Requires DynamoDB redesign (costly)	Add GSI or new table (low-risk)	ShopFlow access patterns still evolving
Max table size	128TB per table (unlimited partitions)	Same per table	No practical difference

■ Single-table design is powerful but complex. Use it when: (1) multiple entity types are always fetched together (user + their orders in one request), (2) access patterns are well-defined and stable, (3) team has DynamoDB expertise. Use multi-table when starting out, patterns are still evolving, or separate teams own different entities.

07-10 DynamoDB Transactions & Conditional Writes

TransactWrite provides ACID transactions across up to 100 items in multiple tables — all succeed or all fail. ShopFlow uses transactions for checkout (cart clear + order create must be atomic) and inventory reservation (stock decrement + event log must not partially succeed).

Checkout Transaction — Cart + Order

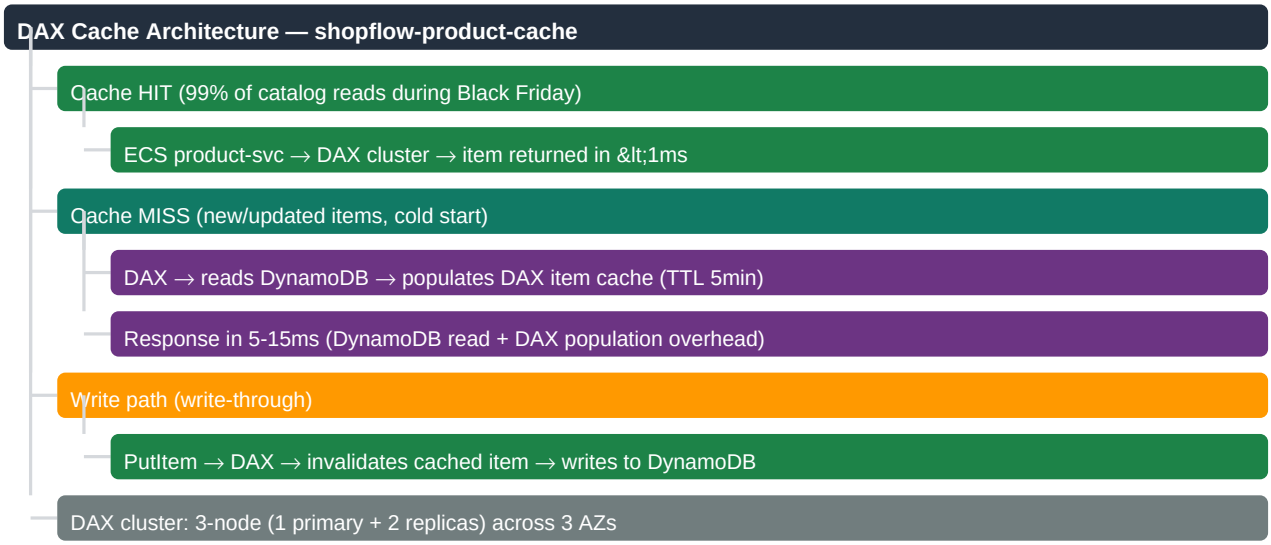
```
dynamo.transactWriteItems(TransactWriteItemsRequest.builder()
.transactItems(
// 1. Create order event (idempotent — fails if already exists)
TransactWriteItem.builder().put(Put.builder()
.tableName("shopflow-order-events")
.item(Map.of(
"orderId", AttributeValue.fromS(orderId),
"sk", AttributeValue.fromS(timestamp + "#evt-001"),
"eventType",AttributeValue.fromS("ORDER_PLACED"),
"userId", AttributeValue.fromS(userId),
"status", AttributeValue.fromS("PENDING"),
"totalAmt", AttributeValue.fromN(total.toString())))
.conditionExpression("attribute_not_exists(orderId)")
.build()).build(),
// 2. Delete cart items (one Delete per item)
TransactWriteItem.builder().delete(Delete.builder()
.tableName("shopflow-cart")
.key(Map.of("userId", AttributeValue.fromS(userId),
"productId", AttributeValue.fromS(p1)))
.build()).build()
).build());
```

Scenario	Tables	Atomicity Guarantee	Cost vs Non-Transactional
Checkout: clear cart + create order	cart + order-events	All-or-nothing across 2 tables	2x WCU (two-phase commit overhead)
Inventory reserve: decrement + log	inventory + order-events	Stock and event always consistent	2x WCU
Idempotent order creation	order-events	attribute_not_exists prevents duplicate	2x WCU — pays for safety

■ Use ConditionExpression="attribute_not_exists(orderId)" for idempotent order creation. If a network timeout causes the Lambda to retry the checkout, the second TransactWrite attempt will fail gracefully (TransactionCanceledException with CONDITIONAL_CHECK_FAILED) rather than creating a duplicate order. Build idempotency into all transaction writes.

07-11 DAX — DynamoDB Accelerator (Sub-millisecond Cache)

DAX is a fully managed, API-compatible in-memory cache for DynamoDB. Sub-millisecond read latency for cache hits vs 1-10ms for direct DynamoDB. ShopFlow uses DAX for the product catalog: 90%+ cache hit ratio, 400K product cache items fit in a 30.5GB r4.large node.



DAX CDK

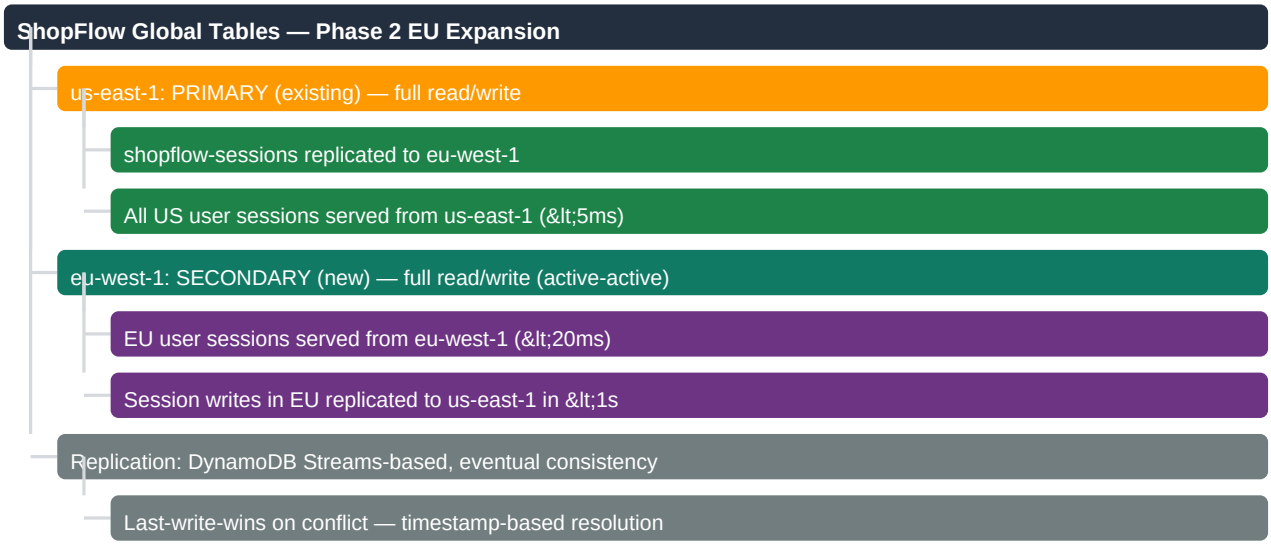
```
const daxCluster = new dax.CfnCluster(this,"ShopFlowDAX",{
clusterName: "shopflow-dax",
nodeType: "dax.r4.large", // 30.5GB — catalog fits in RAM
replicationFactor: 3, // 1 primary + 2 read replicas
iamRoleArn: daxRole.roleArn,
subnetGroupName: daxSubnetGroup.ref,
securityGroupIds: [sgDax.securityGroupId],
sseSpecification: { sseEnabled: true },
});
// Java: DaxClient is a drop-in replacement for DynamoDbClient
DaxClient daxClient = DaxClient.builder()
.region(Region.US_EAST_1)
.endpointConfiguration(new EndpointConfiguration(
daxCluster.getAttrClusterDiscoveryEndpoint(), 8111))
.build();
```

Comparison	DynamoDB Direct	With DAX
Read latency (cache hit)	1-10ms (single-digit)	<1ms (microseconds)
Read latency (cache miss)	1-10ms	5-15ms (DAX populates on miss)
Write-through latency	1-10ms	1-10ms (same)
RCU consumed on cache hit	1 RCU per GetItem	0 RCU — served from DAX RAM
Cost (3-node r4.large)	$0	$0.269/hr × 3 = $590/month

■ DAX is only appropriate for tables where the same items are read repeatedly. Product catalog: 400K products, 80% of reads on the top 5% of popular items — extremely high cache hit ratio. Session table: each session item read by exactly one user — zero cache hits. Never add DAX to session or unique-key lookup tables.

07-12 DynamoDB Global Tables — Multi-Region Active-Active

DynamoDB Global Tables replicate data across multiple regions with last-write-wins conflict resolution. All regions are active for both reads and writes. ShopFlow Phase 2 adds eu-west-1 global table to serve EU users with <20ms session reads (vs <200ms transatlantic).



Global Tables CDK

```
// Enable Global Tables by adding replicas to existing table
const globalSessionTable = new dynamodb.Table(this,"GlobalSessions",{
tableName: "shopflow-sessions",
partitionKey: { name:"userId", type:dynamodb.AttributeType.STRING },
sortKey: { name:"sessionId", type:dynamodb.AttributeType.STRING },
billingMode: dynamodb.BillingMode.PAY_PER_REQUEST,
replicationRegions: ["eu-west-1"], // enable Global Tables replication
timeToLiveAttribute: "expiresAt",
pointInTimeRecovery: true,
});
```

Feature	Single-Region DynamoDB	Global Tables (Multi-Region)
Read latency (US users)	<10ms from us-east-1	<10ms (same)
Read latency (EU users)	150-200ms (transatlantic)	<20ms from eu-west-1
Write latency (EU users)	150-200ms to us-east-1	<20ms to eu-west-1 local
Regional failure (us-east-1)	Full outage	EU region continues serving EU users
Replication lag	N/A	<1 second (typically 100-300ms)
Cost premium	Baseline	+doubles storage + cross-region replication WCU

■ Global Tables use last-write-wins (LWW) conflict resolution based on timestamp. If two regions write to the same item within the replication lag window, one write is silently discarded. For data that must be consistent (inventory counters, financial balances), use conditional writes with optimistic locking (version attribute) or direct writes to the primary region.

07-13 DynamoDB Security & PITR

DynamoDB security layers: KMS CMK encryption at rest for PCI compliance, IAM action-level + condition key policies restricting Lambda to specific operations, VPC Gateway endpoint eliminating NAT costs, and PITR for 35-day point-in-time recovery on all production tables.

Security Layer	Implementation	ShopFlow Config
Encryption at rest	AWS KMS CMK	order-events: alias/shopflow/dynamo-cmk. 90-day rotation.
IAM least privilege	Action + condition keys	Lambda: GetItem/PutItem/UpdateItem on specific table ARN only
VPC Gateway endpoint	dynamo.vpc-endpoint	All DynamoDB traffic via private backbone — no NAT charges

Security Layer	Implementation	ShopFlow Config
PITR	Continuous backup	35-day retention on sessions, cart, order-events
Resource policy	Table-level deny	Deny PutItem from principals outside account (cross-account guard)
CloudTrail data events	PCI audit trail	Log all GetItem/PutItem to S3 for 7-year retention

IAM Policy CDK — Lambda Least Privilege

```
// CDK: grant read/write with automatic least-privilege policy
orderEventsTable.grantReadWriteData(orderLambda);
// Manual fine-grained: restrict to specific operations + condition
orderLambda.addToRolePolicy(new iam.PolicyStatement({
actions: [
"dynamodb:GetItem",
"dynamodb:PutItem",
"dynamodb:UpdateItem",
"dynamodb:Query",
],
resources: [
orderEventsTable.tableArn,
orderEventsTable.tableArn + "/index/*",
],
conditions: {
// LeadingKeys: users can only write items where userId = their Cognito sub
"ForAllValues:StringEquals": {
"dynamodb:LeadingKeys": ["${cognito-identity.amazonaws.com:sub}"]
},
},
}));
```

■ DynamoDB VPC Gateway endpoint is free — no hourly charge, no data transfer charges within the same region. Without it, DynamoDB traffic from ECS tasks exits via NAT Gateway at $0.045/GB. At 10GB/day DynamoDB traffic from ECS: NAT cost = $0.045 × 10 × 30 = $13.50/month. VPC endpoint: $0/month. Enable on every VPC that has DynamoDB-accessing workloads.

07-14 Batch Operations & PartiQL

BatchGetItem retrieves up to 100 items (across tables) in a single API call — more efficient than individual GetItem calls in a loop. BatchWriteItem writes up to 25 items per call. PartiQL provides SQL-compatible syntax for DynamoDB, useful for bulk operations and ad-hoc queries.

BatchGetItem — Load Multiple Cart Items

```
// Fetch 10 product details for cart display (1 API call vs 10 GetItem calls)
var keys = cartItems.stream()
.map(item -> Map.of("productId",
AttributeValue.fromS(item.getProductId())))
.collect(toList());
BatchGetItemResponse resp = dynamo.batchGetItem(
BatchGetItemRequest.builder()
.requestItems(Map.of("shopflow-product-cache",
KeysAndAttributes.builder()
.keys(keys)
.projectionExpression("productId, #n, price, imageUrl")
.expressionAttributeNames(Map.of("#n","name"))
.consistentRead(false) // eventually consistent = 0.5 RCU
.build()))
.build());
// Process: resp.responses().get("shopflow-product-cache")
// Unprocessed keys: resp.unprocessedKeys() — retry with backoff
```

PartiQL — SQL Syntax for DynamoDB

```
// PartiQL SELECT — reads from table (uses KeyCondition internally)
ExecuteStatementResponse r = dynamo.executeStatement(
ExecuteStatementRequest.builder()
.statement("SELECT * FROM \"shopflow-cart\" WHERE userId=?")
.parameters(List.of(AttributeValue.fromS(userId)))
.build());
// PartiQL UPDATE — update item without building UpdateExpression
dynamo.executeStatement(ExecuteStatementRequest.builder()
.statement("UPDATE \"shopflow-sessions\" SET lastActivity=?
WHERE userId=? AND sessionId=?")
.parameters(List.of(AttributeValue.fromS(now()),
AttributeValue.fromS(userId),
AttributeValue.fromS(sessionId)))
.build());
```

API	Max Items	Suitable For	Limitations
GetItem	1 item	Single known key lookup	No batch; no retry logic needed
BatchGetItem	100 items across tables	Cart product details; related entities	Partial responses; retry UnprocessedKeys
Query	Unlimited (paginated)	All items in one PK; range on SK	Only on PK/SK/GSI — no cross-partition
Scan	Unlimited (slow, expensive)	NEVER in production; migration scripts only	Reads ALL partitions; full RCU consumed

■ BatchGetItem may return partial results (UnprocessedKeys) when capacity is insufficient. Always implement retry logic with exponential backoff for UnprocessedKeys. The SDK's RetryMode.ADAPTIVE handles most cases, but verify UnprocessedKeys in the response manually for production code — the adaptive retry does not always retry all unprocessed keys correctly.

07-15 **DynamoDB Monitoring, Alarms & Cost Optimization**

Key DynamoDB metrics: ConsumedRCUs/WCUs, ThrottledRequests, SuccessfulRequestLatency, and UserErrors (400-level). Cost optimization: TTL for storage cleanup (no WCU charged), DAX for read amplification reduction, KEYS_ONLY GSI projections, and right-sizing item sizes.

Metric	Namespace	Alert Threshold	Action Required
ThrottledRequests	AWS/DynamoDB	> 0 for >1 min (Provisioned)	Increase RCU/WCU or switch to On-Demand
SuccessfulRequestLatency (p99)	AWS/DynamoDB	> 10ms for GetItem	Check item size; add DAX if read-heavy
SystemErrors	AWS/DynamoDB	Any non-zero	AWS-side issue; open support ticket
ConsumedRCU (provisioned)	AWS/DynamoDB	> 80% ProvisionedRCU	Scale up RCU or enable Auto Scaling
UserErrors	AWS/DynamoDB	> 0 sustained	Check application — wrong key format, expired TTL

CloudWatch Alarms CDK

```
// Alert on DynamoDB write throttling (on-demand tables should not throttle)
new cloudwatch.Alarm(this,"DynWriteThrottle",{
metric: orderEventsTable.metric("ThrottledRequests",{
dimensionsMap: { Operation:"PutItem" },
statistic: "Sum", period: Duration.minutes(1),
}),
threshold: 1, evaluationPeriods: 3, datapointsToAlarm: 2,
alarmDescription: "DynamoDB PutItem throttled — check capacity",
}).addAlarmAction(new cwa.SnsAction(opsAlertTopic));
```

Cost Optimization Summary

Lever	Mechanism	ShopFlow Estimated Saving
TTL auto-delete	Background deletion — no WCU charged	Sessions: 70% storage reduction (24hr TTL vs perpet
DAX read cache	Repeat reads served from RAM — 0 DynamoDB RCU	Product cache: 90% DynamoDB RCU elimination
KEYS_ONLY GSI projection	Only PK/SK in GSI — minimal storage + write overhead	Cache-invalidation GSI: 80% GSI storage + write amp
On-Demand off-peak	No idle capacity cost at night/weekends	Sessions off-peak (0.1% of peak): zero provisioned R
Compress large items	GZIP items >4KB before PutItem	Order items array: 60% size → 60% RCU/WCU reduc

■ TTL deletions do not consume WCU. However, deleted items DO appear in DynamoDB Streams (with userIdentity.type = "Service"). Always filter TTL deletions in stream Lambda: if (record.getUserIdentity() != null && "Service".equals(record.getUserIdentity().getType())) { return; } — otherwise your order processing Lambda will trigger on session expiry events.

07-BP Best Practices & Anti-Patterns

✓ Design for access patterns first — list every query before designing PK/SK structure

✓ Use On-Demand billing for all production tables with spiky or unpredictable traffic patterns

✓ Enable PITR on all production tables — 35-day point-in-time recovery, zero configuration

✓ Set TTL attribute for auto-expiry — no cron jobs, no WCU cost, automatic storage reduction

✓ Use atomic UpdateItem with ADD for all numeric counter increments and decrements

✓ Add ConditionExpression to all writes where optimistic locking is needed

✓ Use TransactWrite only when cross-table atomicity is genuinely required (checkout, inventory)

✓ Use attribute_not_exists(pk) condition for idempotent create operations

✓ Design sparse GSIs — items without the GSI attribute are excluded automatically

✓ Use INCLUDE or KEYS_ONLY projections on GSIs unless all attributes are needed from the index

✓ Add DynamoDB VPC Gateway endpoint to all VPCs — free, eliminates NAT Gateway traffic cost

✓ Use DAX only for read-heavy tables with high repeat-read ratio (>80% reads, eventual consistent OK)

✓ Enable Streams with bisectOnError=true and DLQ — split batch on failure, never lose events

✓ Use BatchGetItem for fetching multiple known keys — reduce API calls by 10-100x

✓ Handle UnprocessedKeys in BatchGetItem responses — always retry with exponential backoff

✓ Monitor ThrottledRequests per table/operation — alert on any non-zero for provisioned tables

■ Never use Scan in production — reads every item in the table, consumes full table RCU capacity

■ Never rely on TTL for real-time expiry — deletion within 48 hours, not immediate

■ Never use low-cardinality partition keys (date, status, country) — causes hot partitions

■ Never store items > 400KB — DynamoDB hard limit; use S3 for binary/large objects + reference key

■ Never use DynamoDB reserved words as attribute names without ExpressionAttributeNames mapping

■ Do not use String sort keys for numeric ordering — "10" < "9" lexicographically; use zero-padded strings

■ Global Tables LWW conflict resolution silently discards one write — do not use for financial counters

07-QR Quick Reference & Interview Q&A

Concept	Key Value / Rule
Max item size	400 KB per item (including attribute names + values).
1 RCU definition	1 strongly consistent read of ≤4 KB. Or 2 eventually consistent reads of ≤4 KB.
1 WCU definition	1 write of ≤1 KB per second. Items >1 KB round up to next WCU.

Concept	Key Value / Rule
TransactWrite cost	2× WCU/RCU of equivalent non-transactional write (two-phase commit).
TransactWrite max items	100 items across multiple tables per transaction.
TTL deletion timing	Within 48 hours — not instant. Always FilterExpression="expiresAt > :now".
GSI write amplification	Each table write also writes to all GSIs with matching key attributes.
On-Demand cold start	2× previous peak instantly; doubles every 30 min beyond that.
DynamoDB Streams retention	24 hours. Use SQS (7d) or Kinesis (365d) as durable backup.
Max GSIs per table	20 GSIs + 5 LSIs per table.
Hot partition limit	Single partition: ~1,000 WCU/s and ~3,000 RCU/s maximum.
DAX cache TTL	Item cache: 5 minutes. Query/Scan result cache: 60 seconds (configurable).
PITR retention	35 days. Restores to a NEW table. Does not overwrite source table.
VPC endpoint cost	Gateway endpoint for DynamoDB is FREE — no hourly or per-GB charge.
BatchGetItem max items	100 items per call across multiple tables.

Top Interview Questions

Question	Answer
DynamoDB vs Aurora — when each?	DynamoDB: key-value/document patterns, no joins needed, single-digit ms at millions req/s, unpredict
Explain hot partition and prevention	Hot partition: most writes hit one PK value (e.g., date="2024-11-29" — all Black Friday writes to one p
RCU/WCU calculation for ShopFlow?	Session GetItem (0.5KB): rounds to 4KB → 1 RCU (strongly consistent). Cart UpdateItem (0.3KB): rou
How do DynamoDB Streams work?	Streams capture item-level changes in order within each partition. Each shard corresponds to one Dyn
What is a sparse GSI?	Sparse GSI: only items with the GSI partition key attribute present are indexed. Items without the attrib
TransactWrite vs UpdateItem with conditions?	TransactWrite: multiple items across tables, ACID, 2× WCU. Use for checkout (cart + order across 2 ta
DAX vs ElastiCache for DynamoDB?	DAX: DynamoDB-specific, API-compatible (drop-in replacement), write-through, eventually consistent.